CHAPTER 5: SYSTEM DESIGN

5.1 Introduction

This chapter explains how the proposed system was designed before implementation. The design stage was important because it provided a clear picture of how the hardware, software, database and machine learning components would work together as one complete monitoring solution. For this project, the design had to support real-time data collection from the ESP32, database storage, machine learning-based analysis, dashboard visualization and automated alerting while still remaining practical for a prototype environment at HIT. (Proença, Monteiro and Barroso, 2021)

5.2 System Design

- Overall design approach (modular architecture)
  - The proposed system was designed as a modular AI-enhanced IoT solar monitoring solution.
  - The hardware side is responsible for collecting readings from the solar panel environment, while the software side receives, validates, stores, analyzes and displays those readings.
  - This separation of sensing, backend processing, prediction, storage and presentation makes the system easier to understand, maintain and extend.
  - It also supports future scale-up because changes in one layer do not necessarily require the whole system to be redesigned. *(Li, Sun and Li, 2022)*
  - The design is consistent with modern edge–cloud / hybrid smart-energy architectures, where sensing and data collection are kept lightweight while analytics and visualization are centralized for monitoring and decision support. *(Proença, Monteiro and Barroso, 2021; Bonomi et al., 2012)*
- Prototype hardware layer
  - At prototype level, the system is built around a 15W monocrystalline solar panel connected to an ESP32 DevKit V1.
  - The ESP32 reads values from the DHT22 sensor, LDR, and the voltage sensing circuit and sends them to the Next.js application using HTTP requests.
  - Using ESP32-based IoT monitoring for solar systems supports near real-time acquisition at low cost and is widely adopted in similar solar telemetry solutions. *(Hasan, Siddique and Ahmed, 2022)*

- Backend processing and storage layer
  - Once the readings reach the backend, they are validated before processing, which improves reliability by preventing corrupt or incomplete readings from entering the database.
  - The validated readings are stored in SQLite through Prisma, enabling a reliable historical record for comparisons across time (daily profiles, fault frequency, efficiency trend).
  - This database-backed design supports both real-time monitoring and historical analysis, which is necessary for diagnosing gradual losses such as soiling and aging rather than only sudden faults. *(Massi Pavan, Mellit and De Pieri, 2018; Wong et al., 2020)*
- Prediction and analytics layer (efficiency + degradation + fault intelligence)
  - After storage, the readings are used for visualization and machine learning inference, producing outputs such as efficiency prediction and degradation-related indicators.
  - AI techniques are appropriate for PV monitoring because PV behaviour is influenced by multiple non-linear variables (irradiance, temperature, electrical conditions), and ML models can learn operational patterns that simple threshold rules may miss. *(Mellit and Kalogirou, 2008; Garoudja et al., 2017)*
  - ML-based fault detection and diagnosis is also well established in PV monitoring literature, especially when distinguishing fault types like soiling, partial shading, and connection issues. *(Wong et al., 2020; Massi Pavan, Mellit and De Pieri, 2018)*
- Presentation layer (dashboard interaction)
  - The user interacts with the solution mainly through the Dashboard, Predictions, Alerts and Settings pages.
  - The dashboard provides both live monitoring and performance interpretation by combining sensor data trends with predictive outputs (efficiency trend / degradation indicators / health insights), moving the system beyond basic "display-only" monitoring. *(Wong et al., 2020; Mellit and Kalogirou, 2008)*

5.2.1 How will the system work?

- Sensor / data acquisition layer
  - The system begins at the sensor layer.

- The ESP32 collects operational data such as voltage, current, temperature, humidity and irradiance-related values from the connected components.
- The firmware is configured to transmit these readings to the application through the `/api/solar` endpoint at regular intervals, allowing near real-time monitoring of panel performance.

➢ Backend ingestion, validation and storage
- After the data is received by the backend, it is validated to ensure correct format and acceptable ranges before processing.
- Valid readings are then stored in the database, creating a historical record that supports both monitoring and trend-based analysis.

➢ Dashboard visualization (live + historical)
- The stored readings are used in two main ways.
- First, they are displayed directly on the dashboard so that the user can see live values and historical values over time.
- This supports monitoring of performance patterns rather than relying only on isolated readings.

➢ Prediction and analytics (ONNX + prediction logic)
- Second, the readings are processed through the prediction logic and ONNX machine learning models to generate:
  i. fault status
  ii. fault type
  iii. efficiency prediction
  iv. health score
  v. remaining useful life estimates
- This converts raw telemetry into decision-support outputs that can guide maintenance and performance assessment. (Mellit and Kalogirou, 2008; Wong et al., 2020)

➢ Alerting mechanism (Email + WhatsApp)
- The system also includes an alerting mechanism.
- If the readings or predicted outputs show a critical condition such as low voltage, low efficiency, high temperature or likely fault, the system triggers notifications to the responsible user.

- Notifications are delivered through Email and WhatsApp, improving response time and supporting proactive maintenance. (Zonta et al., 2020)

5.3 Solution Architecture – architectural diagram of the proposed solution

The architecture of the proposed solution follows a layered structure. At the bottom is the sensing layer consisting of the solar panel, ESP32 and connected sensors. Above that is the communication layer where the ESP32 sends readings to the backend through HTTP POST requests. The application layer is built with Next.js and contains the main pages and API routes. The data layer uses Prisma with SQLite for storage, while the intelligence layer uses ONNX Runtime to run the trained machine learning models. The notification layer uses Nodemailer for email and WhatsApp alerts and the presentation layer is the browser-based dashboard used by the system user. (Li, Sun and Li, 2022)

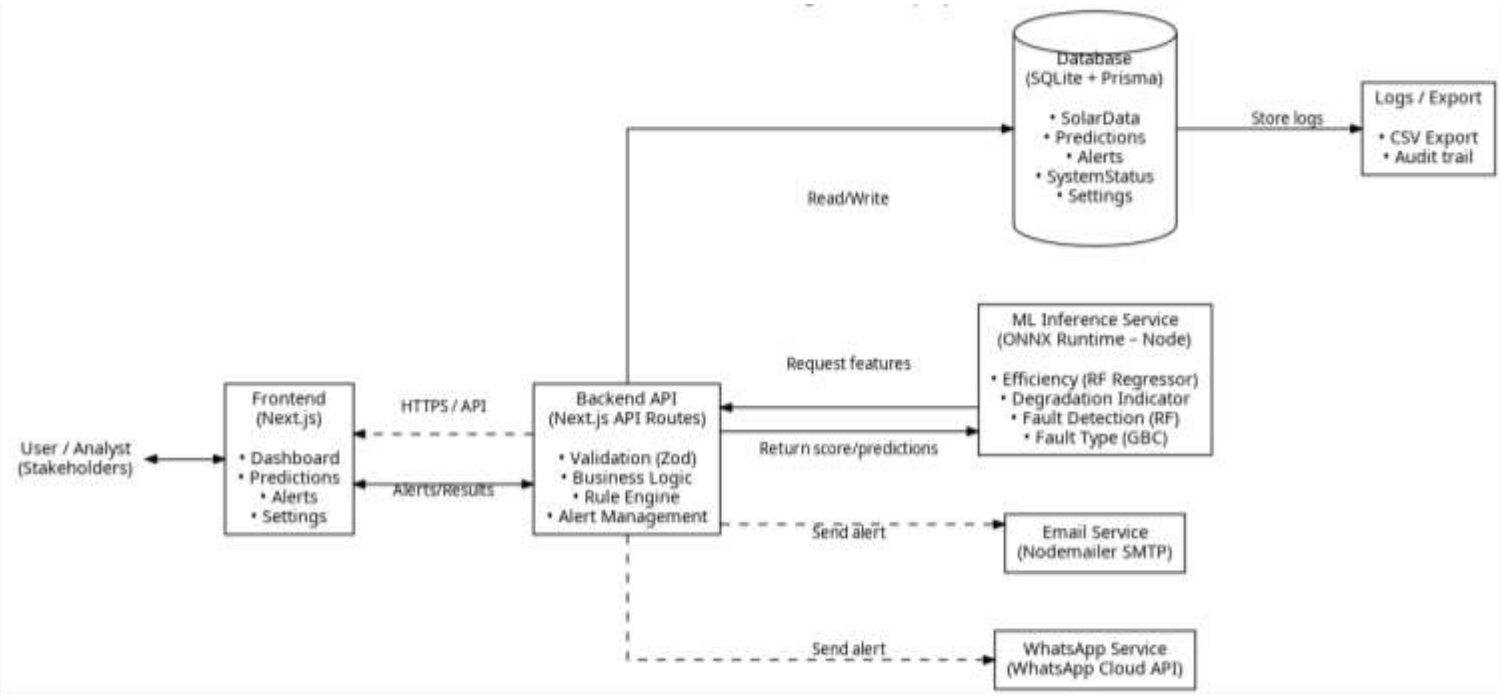

Figure 5.1: Proposed solution architecture

5.3 Database Modelling

- Purpose of the database
  - The database was designed to support continuous storage of solar monitoring information and related system outputs.

- Since the system supports both live monitoring and historical analysis, the database must store repeated readings over time, preserve alert history, and keep system configuration information. *(Wong et al., 2020; Massi Pavan, Mellit and De Pieri, 2018)*

- Choice of database technology
  - A lightweight database approach was considered suitable because the prototype uses SQLite rather than a heavier enterprise database platform.
  - This choice supports a prototype environment where deployment simplicity and frequent inserts are required. *(Proença, Monteiro and Barroso, 2021)*
- Core dataset: sensor readings (time-series storage)
  - The most important data in the system is the stream of sensor readings collected repeatedly over time.
  - Each reading contains values such as voltage, current, power, temperature, humidity and irradiance-related values.
  - These readings form the basis for charts, trend analysis, prediction inputs and fault detection logic. *(Mellit and Kalogirou, 2008; Wong et al., 2020)*
- Prediction outputs and analytics traceability
  - In addition to raw readings, the system stores prediction outputs produced by the inference layer (e.g., efficiency prediction, degradation indicators, fault status/type, confidence and health score).
  - Storing predictions supports traceability: it allows the system to show what the readings were at a given time and what the model concluded from them. *(Wong et al., 2020; Garoudja et al., 2017)*
- Alerts and event history
  - The system keeps information about generated alerts so stakeholders can review when events occurred, what triggered them, and their severity.
  - This supports proactive maintenance by providing an audit trail of abnormal conditions and response actions. *(Zonta et al., 2020)*
- Configuration and settings storage
  - The system also stores configuration values such as monitoring thresholds and notification settings (Email/WhatsApp) to allow controlled changes without editing code.

- Keeping settings separate supports scalability and easier long-term management of the monitoring workflow. *(Li, Sun and Li, 2022; Bonomi et al., 2012)*

5.4.1 E-R Diagram

The E-R diagram for the proposed solution is centered on the major entities used by the system. SensorReading stores the live data coming from the ESP32. PredictionResult stores the analytical output derived from the readings. Alert stores warning messages triggered by abnormal conditions. PanelSetting and SystemSetting store configuration values that control how the system behaves. This separation keeps the monitoring records and system configuration organized.

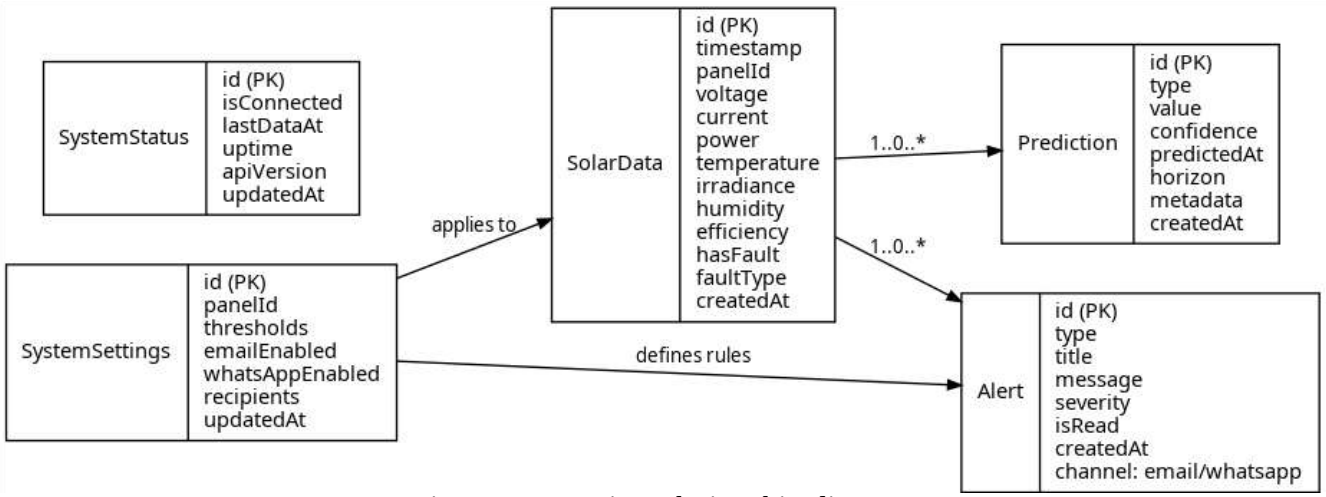

Figure 5.2: Entity relationship diagram

5.4.2 Data Dictionary

SOLAR_DATA (Telemetry Readings)

Field	Data Type	Description	Key/Constraints
id	TEXT	Unique identifier for each reading	PK, unique, not null
timestamp	DATETIME	Time the reading was captured (device or server time)	not null
panelId	TEXT	Identifier of the monitored panel (e.g., PANEL_01)	indexed (recommended)
voltage	REAL	Panel voltage reading (V)	≥ 0
current	REAL	Panel current reading (A)	≥ 0
power	REAL	Calculated/recorded power (W = V×I)	≥ 0
temperature	REAL	Panel/environment temperature (°C)	expected range depends on sensor
humidity	REAL	Relative humidity (%)	0–100
irradiance	REAL	Irradiance proxy or derived irradiance value	≥ 0
efficiency	REAL	Efficiency estimate (%) or computed KPI	0–100 (typical)
hasFault	BOOLEAN	Flag indicating fault detected for this reading	true/false
faultType	TEXT	Fault label/class (e.g., soiling, shading, voltage_anomaly)	nullable
createdAt	DATETIME	Record creation timestamp in DB	not null

PREDICTION (AI inference outputs)

Field	Data Type	Description	Key/Constraints
id	TEXT	Unique identifier for prediction output	PK, unique
type	TEXT	Prediction type (efficiency_forecast, degradation_rate, fault_probability, health_score, etc.)	not null
value	REAL	Numeric prediction value for the given type	not null
confidence	REAL	Confidence/uncertainty score for the prediction	0–1 or 0–100 (choose one consistently)
predictedAt	DATETIME	Time prediction was generated	not null
horizon	TEXT	Forecast horizon (e.g., 24h, 7d, 30d)	nullable
metadata	TEXT	Extra info (e.g., feature summary, model version, JSON blob)	nullable
createdAt	DATETIME	Record creation timestamp	not null
solarDataId	TEXT	Reference to the telemetry record used	FK → SOLAR_DATA.id (recommended)

ALERT (Fault and warning events)

Field	Data Type	Description	Key/Constraints
id	TEXT	Unique identifier for an alert event	PK, unique
type	TEXT	Alert category (fault_detected, high_temperature, low_voltage, low_efficiency, connection_lost)	not null
title	TEXT	Short alert heading displayed to users	not null
message	TEXT	Detailed message including readings and recommended action	not null
severity	TEXT	Severity level (info, warning, critical)	not null
isRead	BOOLEAN	Whether the alert has been acknowledged/read	default false
channel	TEXT	Notification channel used (email, whatsapp, both)	not null
createdAt	DATETIME	Time alert was created	not null
solarDataId	TEXT	Telemetry record associated with this alert	FK → SOLAR_DATA.id (recommended)
settingsId	TEXT	Settings profile used (thresholds/recipients)	FK → SYSTEM_SETTINGS.id (recommended)

SYSTEM_STATUS (Connectivity and uptime)

Field	Data Type	Description	Key/Constraints
id	TEXT	Unique identifier for status record	PK
isConnected	BOOLEAN	True if device has recently sent data	not null
lastDataAt	DATETIME	Timestamp of last telemetry received	nullable (if none yet)
uptime	INTEGER	System uptime or device uptime (seconds)	≥ 0
apiVersion	TEXT	API/app version string	nullable
updatedAt	DATETIME	Timestamp of latest status update	not null

SYSTEM_SETTINGS (Thresholds and notification configuration)

Field	Data Type	Description	Key/Constraints
id	TEXT	Unique identifier for settings record	PK
panelId	TEXT	Panel identifier these settings apply to	not null
ratedPower	REAL	Rated panel power (W)	≥ 0
ratedVoltage	REAL	Rated voltage (V)	≥ 0
ratedCurrent	REAL	Rated current (A)	≥ 0
dataInterval	INTEGER	Telemetry send interval (seconds)	≥ 1
thresholds	TEXT	Threshold configuration (JSON/string: temp limit, efficiency limit, etc.)	not null
emailEnabled	BOOLEAN	Enable/disable email alerts	true/false
whatsAppEnabled	BOOLEAN	Enable/disable WhatsApp alerts	true/false
emailRecipients	TEXT	Recipient list (comma-separated or JSON array)	nullable
whatsAppRecipients	TEXT	WhatsApp target numbers/IDs (comma-separated or JSON)	nullable
updatedAt	DATETIME	Last updated time for settings	not null

In this system, all confidence values are reported as percentages (0–100%) for consistency and easier interpretation by stakeholders.

5.4.3 Database Schema

SolarData Schema *(stores telemetry from ESP32)*

```
{
  "id": "string",
  "timestamp": "datetime",
  "panelId": "string",
  "voltage": 0.0,
  "current": 0.0,
  "power": 0.0,
  "temperature": 0.0,
  "humidity": 0.0,
  "irradiance": 0.0,
  "efficiency": 0.0,
  "hasFault": false,
  "faultType": "string",
  "createdAt": "datetime"
}
```

Prediction Schema ***(stores AI/ONNX outputs from inference)***

```
{
  "id": "string",
  "solarDataId": "string",
  "type": "string",
  "value": 0.0,
  "confidence": 0.0,
  "predictedAt": "datetime",
  "horizon": "string",
  "metadata": "string",
  "createdAt": "datetime"
}
```

Alert Schema ***(stores triggered fault/warning events + channels used)***

```
{
  "id": "string",
  "solarDataId": "string",
  "type": "string",
  "title": "string",
  "message": "string",
  "severity": "info/warning/critical",
  "status": "open/resolved",
  "channel": "email/whatsapp/both",
  "isRead": false,
  "createdAt": "datetime"
}
```

SystemStatus Schema *(connectivity + system health snapshot)*

```
{
  "id": "string",
  "isConnected": true,
  "lastDataAt": "datetime",
  "uptime": 0,
  "apiVersion": "string",
  "updatedAt": "datetime"
}
```

SystemSettings Schema *(thresholds + Email + WhatsApp configuration)*

```
{
  "id": "string",
  "panelId": "string",
  "thresholdValues": "string",

  "smtp_host": "string",
  "smtp_port": 587,
  "smtp_user": "string",
  "smtp_pass": "string",
  "alert_email_from": "string",
  "alert_email_to": "string",

  "whatsapp_access_token": "string",
  "whatsapp_phone_number_id": "string",
  "whatsapp_recipient_phone": "string",
  "whatsapp_api_version": "string",
  "whatsapp_template_name": "string",

  "updatedAt": "datetime"
}
```


5.5 Algorithm Design

This section explains the logic used to convert raw sensor readings into efficiency/degradation predictions, fault decisions, and actionable outputs on the dashboard. The design follows a pipeline approach: ingest → validate → feature engineering → model inference → post-processing → storage → visualization → alerts. (Mellit and Kalogirou, 2008; Wong et al., 2020)

5.5.1 Inputs to the algorithm

- The algorithm processes a telemetry record containing:
  - Voltage (V), Current (A), Temperature (°C), Irradiance (proxy)
  - Timestamp and Panel ID
- Derived values are computed where required:

- Power = Voltage × Current
- Efficiency-related ratios and stability indicators (Mellit and Kalogirou, 2008)

5.5.2 Pre-processing and validation

- Schema validation
  - Confirm required fields are present and numeric (e.g., voltage/current not null).
  - Reject invalid readings (e.g., negative voltage/current, impossible ranges).
- Normalization/cleaning
  - Handle missing optional values (e.g., humidity may be null).
  - Clamp extreme sensor noise if necessary to protect inference stability. (Wong et al., 2020)

5.5.3 Feature engineering (model inputs)

- The following features are computed to improve model performance: Power (W)
  - Voltage–Current ratio and/or Power–Irradiance ratio
  - Temperature-adjusted indicators (since PV efficiency reduces as temperature increases)
  - Rolling-window statistics (if historical data exists): mean, moving variance, short-term trend slope
- Feature engineering is important because PV behaviour is non-linear and influenced by combined environmental/electrical factors. (Mellit and Kalogirou, 2008; Massi Pavan, Mellit and De Pieri, 2018)

5.5.4 Model selection and responsibilities

- The system uses separate models for different tasks to improve accuracy and interpretability:
  - Efficiency prediction (Regression): Random Forest Regressor
    Chosen for robustness to noisy sensor data and non-linear relationships. (Mellit and Kalogirou, 2008)
  - Fault detection (Binary classification): Random Forest Classifier
    Outputs hasFault = true/false based on anomaly/fault patterns. (Wong et al., 2020)
  - Fault type classification (Multi-class): Gradient Boosting Classifier
    Predicts fault categories such as soiling, partial shading, connection issues, aging, etc. (Garoudja et al., 2017; Wong et al., 2020)

- Models are exported to ONNX and executed using ONNX Runtime in the backend for fast, consistent inference. (Wong et al., 2020)

5.5.5 Inference and post-processing logic

- Step 1: Efficiency prediction
  - Predict expected efficiency from the engineered feature vector.
- Step 2: Fault detection
  - Predict hasFault.
- Step 3: Fault type
  - If hasFault = true, run the multi-class model to predict faultType.
- Step 4: Confidence scoring (percentage)
  - Confidence is stored as a percentage (0–100%).
  - For classification, confidence is derived from the model's class probability (e.g., max class probability × 100).
  - For regression, confidence may be derived from model stability/variance or rule-based uncertainty based on sensor noise and data sufficiency. (Wong et al., 2020)
- Step 5: Health score and remaining useful life
  - Health score is computed from efficiency level, fault history, and stability indicators.
  - Remaining useful life estimation is produced only when sufficient historical trend exists; otherwise, the system returns "insufficient data" to avoid misleading results. (Zonta et al., 2020)

5.5.6 Alert decision rules (Email + WhatsApp)

- Alerts are triggered when either sensor readings or predictions exceed critical thresholds, for example:
  - Low voltage
  - Low efficiency
  - High temperature
  - Fault detected / high fault probability
- When an alert condition is met
  - An alert record is created and stored in the database.
  - Notifications are dispatched through Email and WhatsApp

- This supports proactive maintenance by reducing reaction time and encouraging early intervention rather than waiting for major failure. (Zonta et al., 2020)

5.5.7 Algorithm pseudocode

Start system and initialize hardware
Read sensor data from ESP32
Send data to /api/solar
Validate incoming values
Calculate power and related features
Store readings in database
Run machine learning prediction
Generate health and efficiency analysis
Check alert thresholds
Send email alert if condition is critical
Send WhatsApp alert if condition is critical
Display latest information on dashboard
Repeat after configured send interval

5.6 Interface Design

The system interface was designed to make monitoring simple and easy to follow. Since the main users are expected to be technical staff or maintenance personnel, the interface had to show important information clearly without forcing the user to search too deeply. This is why the application is divided into a few main pages instead of one crowded screen.

5.6.1 Dashboard Interface

The Dashboard is the main monitoring page.

It displays live cards for:

- Voltage
- Current
- Power
- Temperature

- Charts showing trends over time

It also shows connection status and panel details.

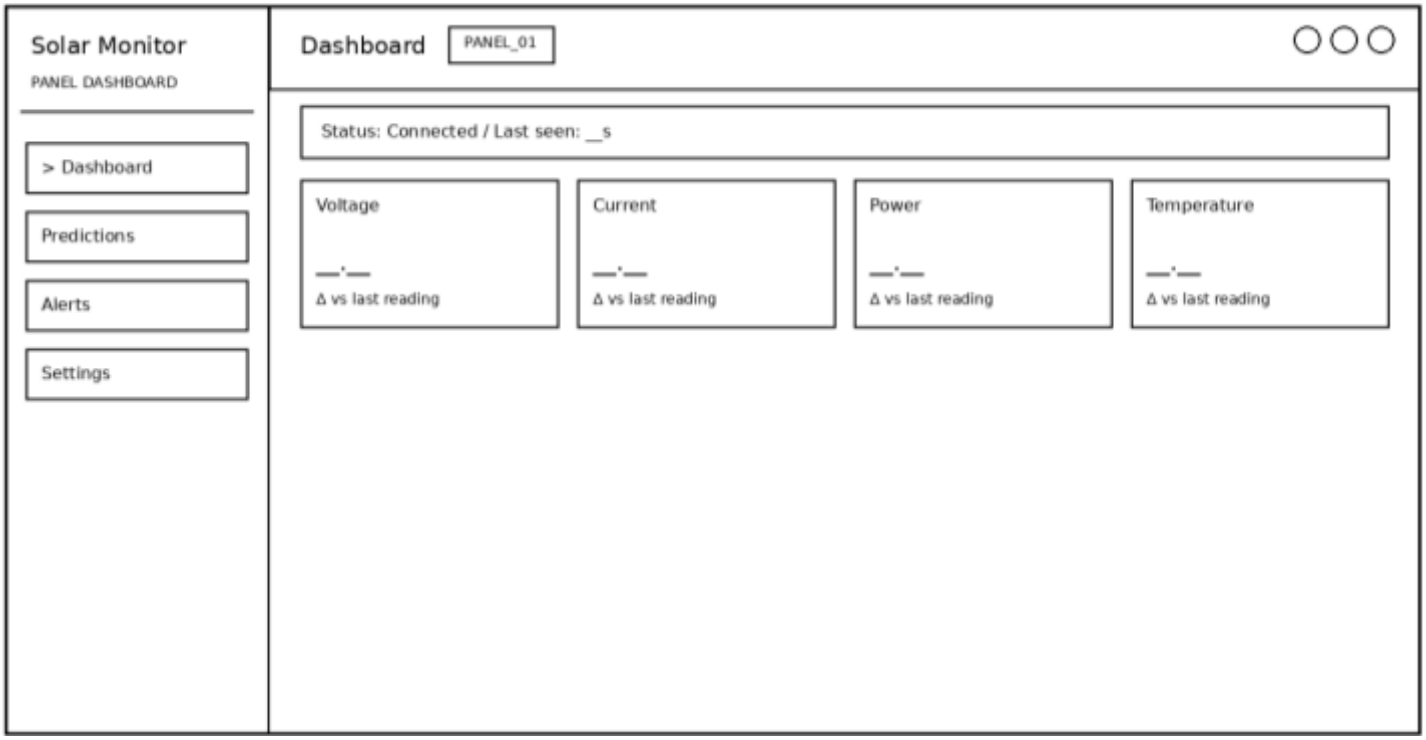

Figure 5.6.1 Dashboard Wireframe (Status + KPI Cards)

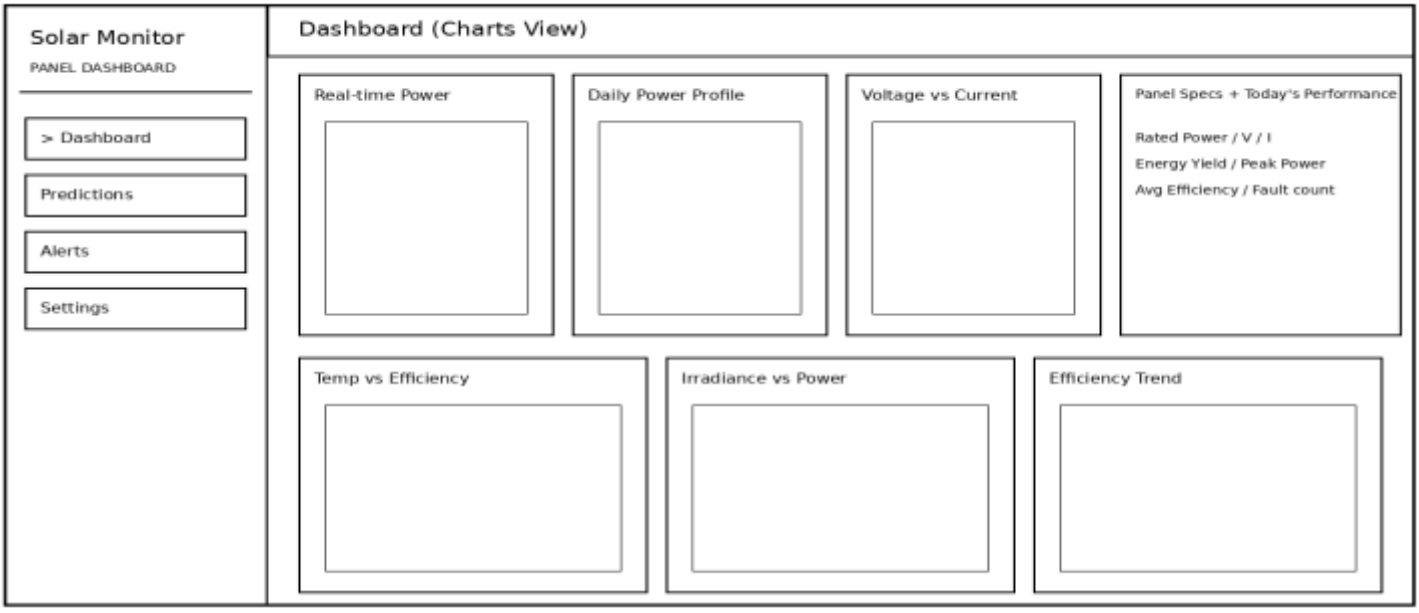

Figure 5.6.2: Dashboard Wireframe (Charts + Performance Summary)

5.6.2 Predictions Interface

The Predictions page focuses on analytics and includes the health score gauge, efficiency chart, fault prediction output, lifetime estimator and degradation analysis.

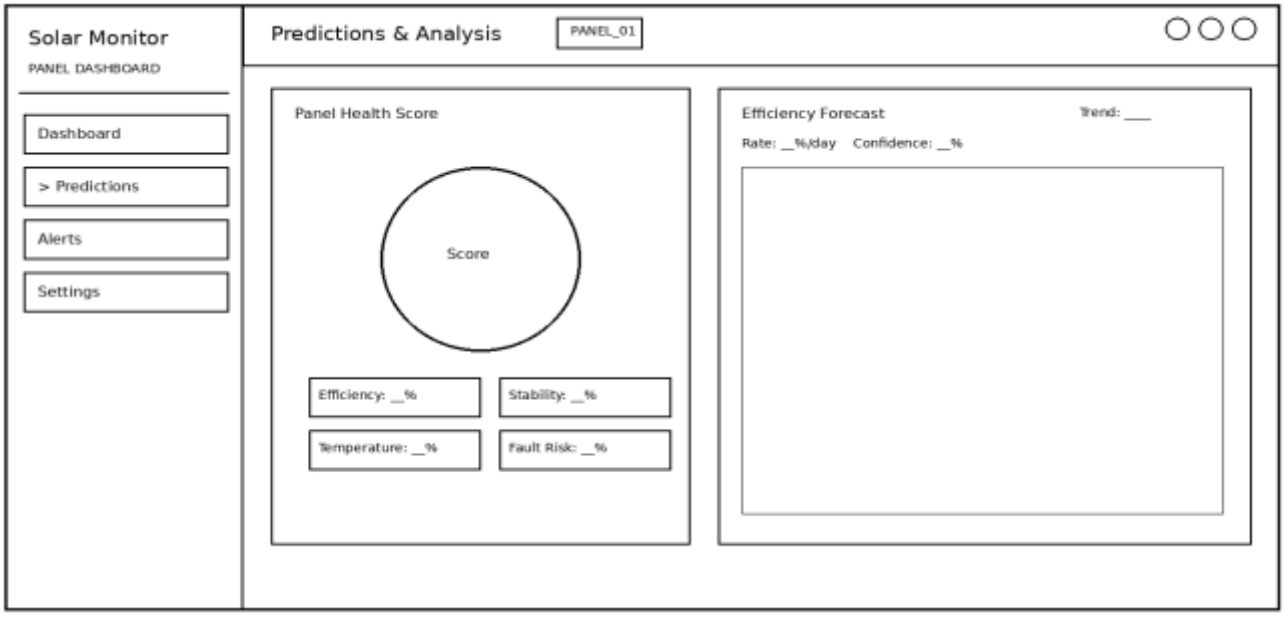

Figure 5.6.3: Predictions Wireframe(Health Score + Efficiency)

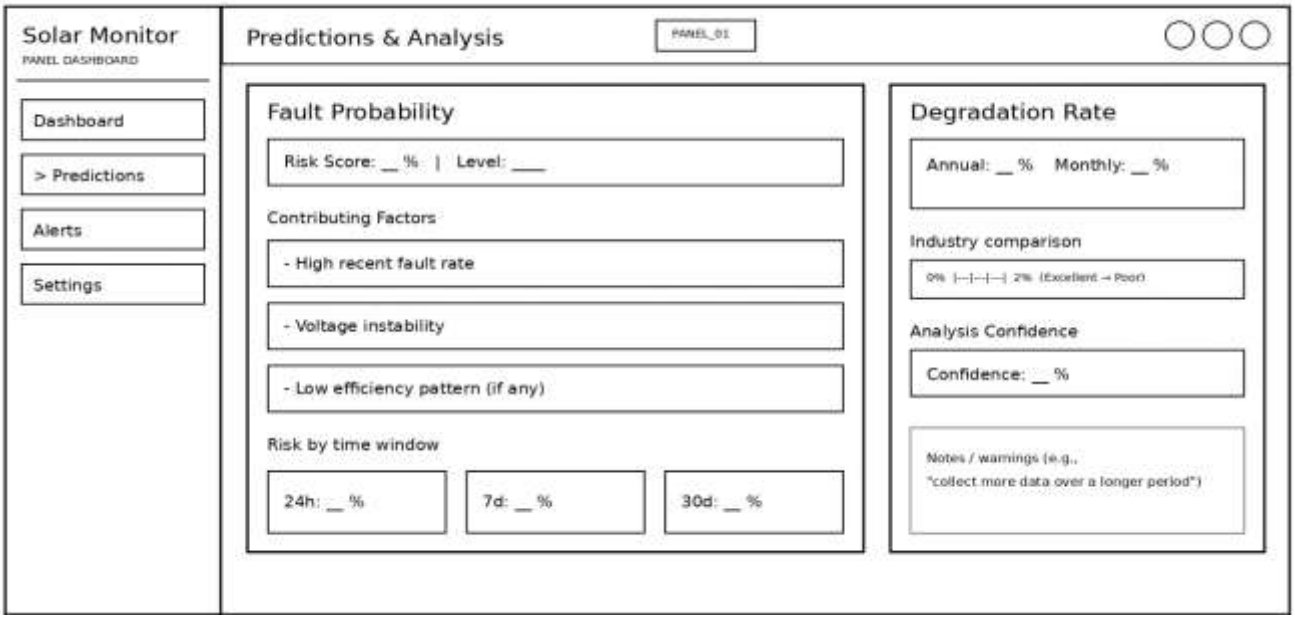

Figure 5.6.4: Predictions Wireframe (Fault Probability + Degradation rate)

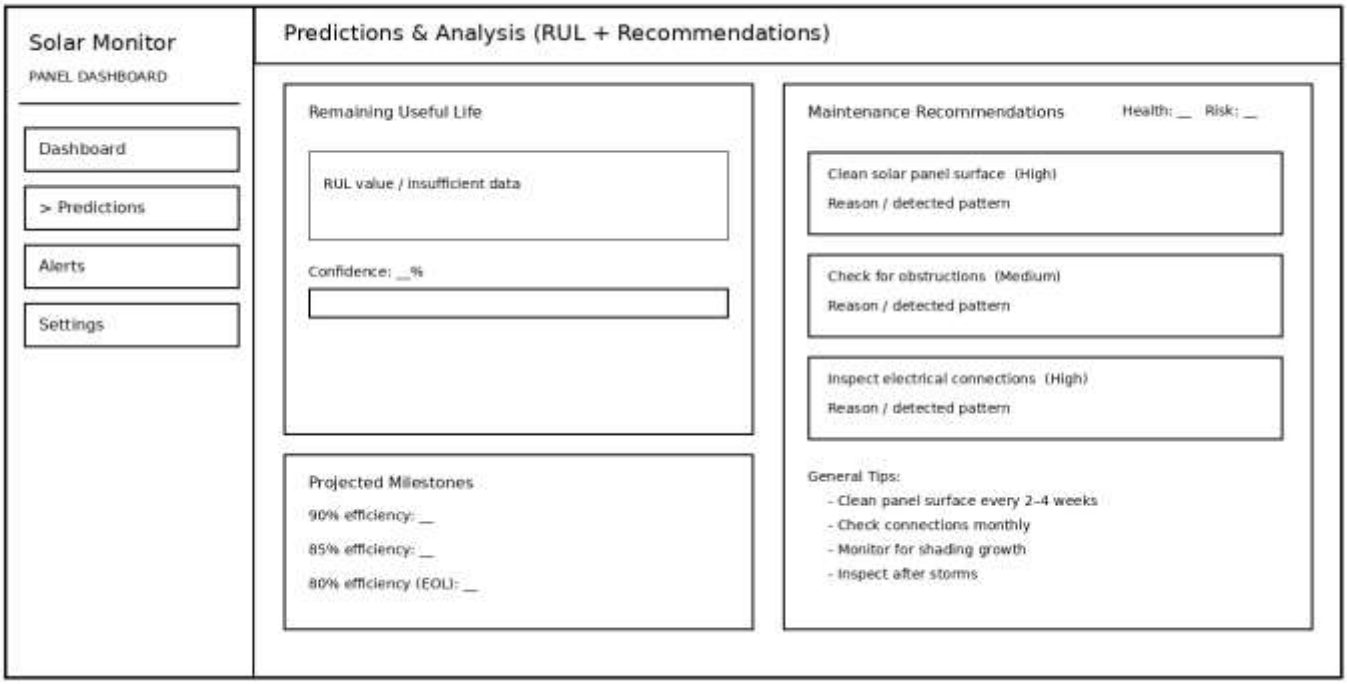

Figure 5.6.5: Predictions Wireframe (RUL + Maintenance Recommendations)

5.6.3 Alerts Interface

The Alerts page stores historical alerts and shows their severity and time.

Figure 5.6.6: Alerts Wireframe (System Alerts List)

5.6.4 Settings Interface

The Settings page is used for email configuration, alert thresholds, panel settings and system information.

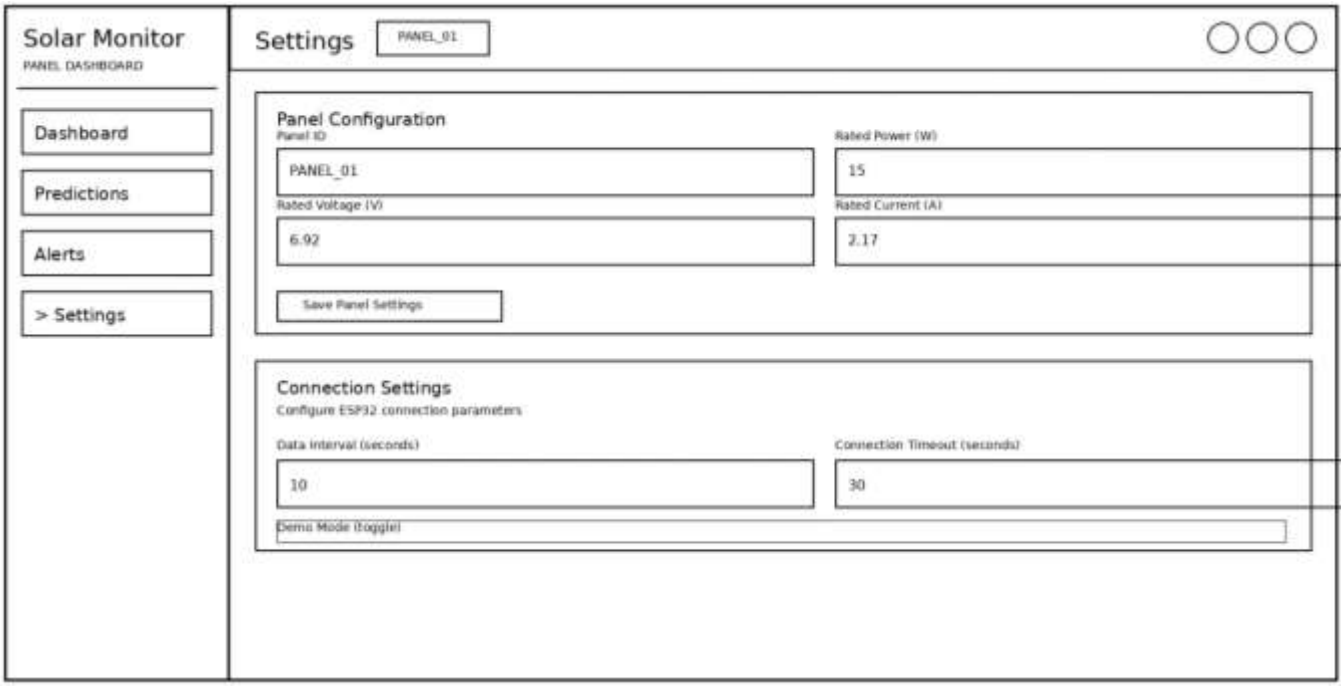

Figure 5.6.7: Settings Wireframe (Panel + Connection Configuration)

Figure 5.6.8: Settings Wireframe (Notifications)

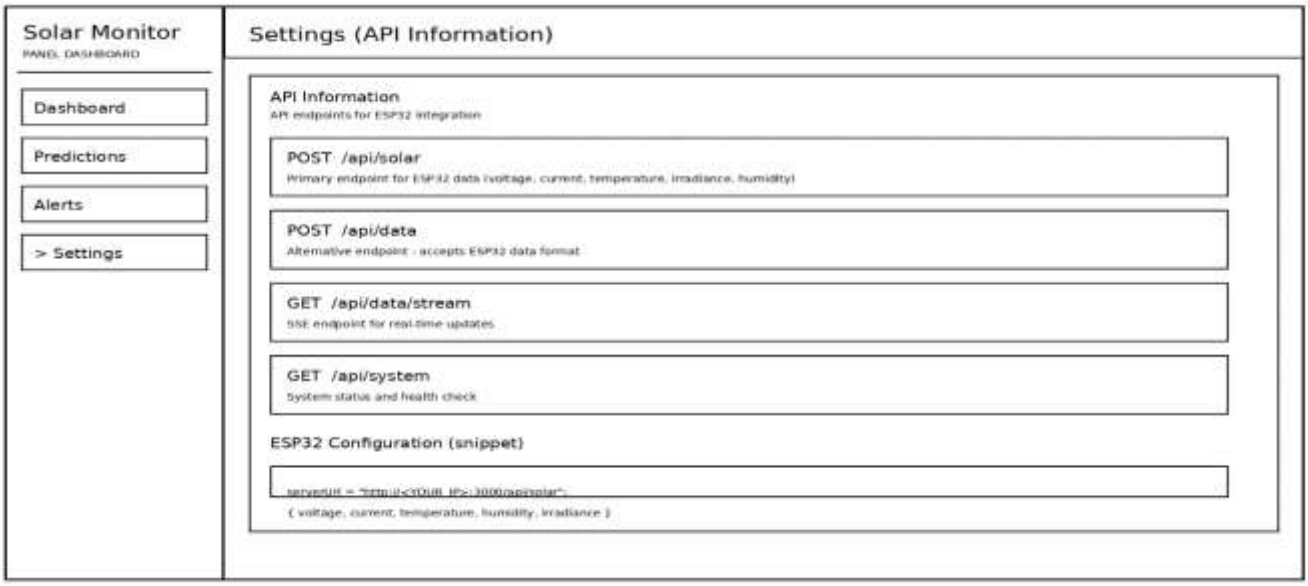

Figure 5.6.9: Settings Wireframe (API Information + ESP32 Snippet)

5.7 Security Design

The security design of the system focuses on protecting monitoring data, configuration values and alert settings from misuse. Since the prototype stores readings in a database and uses SMTP credentials for email and WhatsApp alerts, sensitive configuration values should not be hard-coded directly into the application. Instead, environment variables are used to store database and SMTP details. Incoming sensor values are also validated before they are stored or passed into the prediction logic. These measures help reduce errors, protect configuration information and improve the reliability of the monitoring process.

At deployment stage, stronger controls can be added, such as dashboard authentication, HTTPS, role-based access control and audit logging. Even though the current system is a prototype, these considerations are important because the solution is intended to be extendable into a wider institutional deployment in future. (Dastjerdi and Buyya, 2016; Proença, Monteiro and Barroso, 2021)

5.8 Conclusion

This chapter explained the design of the proposed AI-enhanced IoT solar monitoring system. It showed how the system was structured from the sensing layer up to the dashboard and alerting layer. It also explained the database model, algorithm flow, interface structure and security considerations that support the solution. The next chapter focuses on implementation and testing of the prototype.

CHAPTER 6: IMPLEMENTATION AND TESTING

6.1 Introduction

This chapter explains how the proposed system was implemented and how it was tested. The implementation phase brought together the hardware, web application, database and machine learning components described in the previous chapter. The purpose of testing was to check whether the system could actually perform the tasks it was designed for, such as receiving sensor data, storing it, displaying it on the dashboard, generating predictions and sending alerts.

- The implementation was validated through end-to-end workflow tests, starting from ESP32 telemetry submission and ending at dashboard visualization and alert dispatch.
- Testing was done using:
    - live ESP32 sensor transmission and
    - simulated telemetry injection (demo mode / test payloads) to reproduce fault scenarios consistently.
- Results were evaluated based on:
    - correctness of stored readings and calculated fields (e.g., power and efficiency),
    - correctness and consistency of predictions,
    - responsiveness of the dashboard stream,
    - correctness of alert triggering and notification delivery.

6.2 Coding Strategy

The coding strategy used in this project was modular. Instead of developing the whole system as one large block, the prototype was divided into separate sections. There was a firmware part for the ESP32, a web application part for the dashboard and API routes, a database part for data storage and a machine learning part for training and exporting models. This approach made development easier because each module could be worked on independently and later connected to the others.

- Firmware development was done in Arduino IDE using C/C++ for ESP32 data collection.
- The web application was developed in Next.js with React, TypeScript and Tailwind CSS.
- Database interaction was handled through Prisma with SQLite.
- Machine learning models were trained in Python and exported to ONNX for inference.
- Alerting was implemented using Nodemailer with SMTP configuration and WhatsApp Cloud API for WhatsApp messaging.

- Separation of concerns: firmware handles acquisition/transmission, backend handles validation/storage/inference, UI handles visualization.
- Reusability: shared utilities were used for calculations (power, efficiency, derived features) to prevent inconsistent results between charts and prediction logic.
- Deployment practicality: the stack was chosen so the prototype can run locally (SQLite + Next.js) without requiring external cloud infrastructure during testing.

6.3 Coding Review

Code review was necessary because the system contains multiple connected modules that depend on each other. The firmware had to send the correct values, the backend had to receive and validate them properly, the database had to store them correctly and the prediction logic had to use them consistently. For that reason, review focused on correctness, readability, modularity and consistency across the different parts of the system.

Code review focus areas

- **Firmware payload correctness**
  - Confirmed JSON keys and data types match backend expectations (voltage, current, temperature, irradiance, humidity optional).
  - Confirmed data interval timing matches configured send interval (e.g., every 10 seconds).
- **Backend input validation**
  - Verified invalid values are rejected (e.g., missing keys, negative voltage/current).
  - Verified calculated fields (power = V×I) are computed consistently.
- **Database integrity**
  - Confirmed readings insert correctly with timestamps and correct panelId.
  - Confirmed indexing supports time-ordered retrieval (latest reading and daily aggregation).
- **ONNX inference integration**
  - Verified ONNX models load without runtime error.
  - Verified feature vector ordering matches what models were trained on.
  - Confirmed prediction outputs are persisted and displayed correctly.
- **Alerting correctness (Email + WhatsApp)**

- Verified alert rules trigger correctly when thresholds are exceeded.
- Verified both notification channels can be invoked and tested independently.

6.4 Types of Testing and Results

The prototype was examined using both functional and non-functional testing. Functional testing focused on whether the system could do what it was supposed to do. Non-functional testing focused on qualities such as responsiveness, usability, maintainability and reliability. Because this is a prototype, the emphasis was on confirming that the main monitoring workflow could operate correctly from end to end in the development environment.

6.4.1 Functional Testing

Functional testing checked the major features of the prototype, including receiving readings from the ESP32, saving the readings in the database, displaying them on the dashboard, generating predictions, storing alerts and sending a test email. During prototype verification, the core monitoring workflow behaved as expected in the local environment.

Table 6.1: Functional test cases and results

Test ID	Feature Tested	Test Input/Scenario	Expected Result	Actual Result	Status
FT-01	Telemetry ingestion	ESP32 POST to /api/solar	API returns 200 OK	200 OK received	Pass
FT-02	Validation	Missing voltage field	Reject request (400)	400 returned	Pass
FT-03	Power calculation	V=6.0, I=1.0	Power stored ≈ 6.0W	Stored 6.0W	Pass
FT-04	DB storage	Send 10 readings	10 rows stored in SolarData	10 rows present	Pass
FT-05	Latest reading endpoint	GET /api/data/latest	Returns most recent record	Correct record returned	Pass

Test ID	Feature Tested	Test Input/Scenario	Expected Result	Actual Result	Status
FT-06	Dashboard update	SSE subscribe /api/data/stream	UI updates without refresh	Updates observed	Pass
FT-07	Prediction generation	Normal readings	Prediction stored + shown	Displayed on Predictions	Pass
FT-08	Alert record creation	Force high temperature	Alert stored in Alert table	Alert stored	Pass
FT-09	Email alert	Trigger critical condition	Email sent to configured recipient	Delivered	Pass
FT-10	WhatsApp alert	Trigger critical condition	WhatsApp message delivered	Delivered	Pass

Functional testing confirmed that the core workflow works end-to-end: data capture → API ingestion → validation → database storage → prediction generation → dashboard visualization → alerting.

6.4.2 Non-Functional Testing

Non-functional testing focused on how well the system behaved rather than only what it did. The main areas considered were performance, usability, maintainability and reliability. The prototype was designed to receive data every 10 seconds, refresh the dashboard automatically and provide a simple interface that could be understood by a technical user.

Table 6.2: Non-functional tests

Test Area	What was checked	Measurement/Observation	Status
Performance	Dashboard refresh	SSE stream updates visible within ~1–3 seconds after insert	Pass
Throughput	Insert frequency	System handles 10s interval without backlog	Pass
Reliability	Continuous run	Ran for 10 minutes without crash	Pass
Usability	Navigation clarity	Sidebar pages are distinct (Dashboard/Predictions/Alerts/Settings)	Pass
Maintainability	Modular structure	Firmware/backend/UI separated for isolated changes	Pass
Security (prototype)	Sensitive values	SMTP/WhatsApp credentials stored in env variables (not hard-coded)	Pass

Non-functional testing indicates the prototype is responsive enough for near real-time monitoring and remains stable under the configured telemetry interval. As a prototype, results are based on local testing, but the same architecture supports scaling with stronger infrastructure.

6.4.3 Test Data and Tools Used

- Live ESP32 readings during prototype operation.
- Simulation/test payloads to reproduce faults (soiling/shading/voltage anomaly/high temperature).
- Prisma Studio used to verify inserted rows.
- Browser inspection used to confirm API responses and UI updates.
- Email and WhatsApp test runs to confirm notification delivery.

6.5 Test Cases

Sample test cases used during system verification are shown below. They cover valid and invalid input, dashboard access, prediction display and alert triggering.

Table 6.2: Sample Functional Test Cases

Test ID	Feature Tested	Test Input / Scenario	Expected Result	Actual Result	Status
TC-01	Sensor data submission	ESP32 sends valid JSON to POST /api/solar	API returns **200 OK**	200 OK received	Pass
TC-02	Input validation	Missing field (e.g., voltage)	Request rejected (**400**)	400 returned	Pass
TC-03	Range validation	Negative voltage/current	Request rejected or logged	Rejected/logged	Pass
TC-04	Power calculation	V=6.0, I=1.0	Stored power ≈ 6.0W	Stored correctly	Pass
TC-05	Database persistence	Send 10 readings at interval	10 rows stored in **SolarData**	Rows confirmed	Pass
TC-06	Latest reading retrieval	GET /api/data/latest	Returns most recent record	Correct record returned	Pass
TC-07	Dashboard display	Open Dashboard page	Latest telemetry visible	Visible	Pass
TC-08	Predictions display	Open Predictions tab after data arrives	Efficiency/health outputs visible	Visible	Pass

Test ID	Feature Tested	Test Input / Scenario	Expected Result	Actual Result	Status
TC-09	Alert generation	Simulate high temperature / low efficiency	New record appears in **Alert** table/page	Alert created	Pass
TC-10	Email alert	Trigger critical condition	Email sent to configured recipient	Delivered	Pass
TC-11	WhatsApp alert	Trigger critical condition	WhatsApp message delivered to recipient	Delivered	Pass
TC-12	Alert acknowledgement	Mark alert as read/acknowledged	Status changes to read	Updated	Pass

These test cases confirm that the system functions correctly across the full workflow: telemetry ingestion, validation, database storage, dashboard visualization, prediction presentation, alert creation, and notification delivery through Email and WhatsApp.

6.6 Levels of Testing and Results

Testing was considered at four levels: unit testing, integration testing, validation testing and system testing. These levels were useful because the prototype contains several connected modules rather than one isolated program.

6.6.1 Unit Testing

Unit testing focused on individual parts of the system. In this project, this included testing the ESP32 data payload, API route handling, data calculations, database functions and email alert logic separately. The main purpose at this level was to make sure that each component worked correctly on its own before it was connected to the rest of the prototype.

- Verified ESP32 payload structure matched API expectations (key names and numeric types).
- Verified backend calculation function returns correct power values for known V and I.

- Verified database insert and retrieval functions return expected rows and ordering.
- Verified email and WhatsApp send functions trigger correctly using test endpoints/config.

6.6.2 Integration testing

Integration testing checked whether the connected modules worked properly together. The most important integration flow for this project was ESP32 to /api/solar to database to prediction logic to dashboard and alerts. This level was important because even when individual modules work correctly on their own, problems can still appear when data begins to move between them. The integration flow operated successfully in the local prototype environment.

- Confirmed end-to-end flow: ESP32 → /api/solar → SQLite → predictions → dashboard.
- Confirmed alerts flow: prediction/threshold breach → Alert stored → email + WhatsApp dispatch.
- Confirmed dashboard pages pull correct data (latest + history) without manual DB edits.

6.6.3 Validation testing

Validation testing was used to check whether the system met the original objectives of the project. For this study, that meant checking whether the prototype truly behaved like an AI-enhanced solar monitoring solution and not just a simple dashboard. The validation therefore focused on real-time data capture, prediction support, automated alerts and usefulness of information for monitoring and maintenance. (Zonta et al., 2020)

- Confirmed system behavior aligns with objectives: real-time monitoring + AI predictions + automated alerts.
- Confirmed the dashboard supports maintenance decisions through health score, fault probability and recommendations.

6.6.4 System Testing

System testing considered the full prototype as one complete solution. This included the hardware setup, the dashboard, the alerts, the predictions and the supporting configuration. At this level, the goal was to observe whether the prototype behaved as one working solar monitoring system under normal use. This is the level most relevant to the final demonstration because it reflects what the end user would actually experience.

- Ran the prototype as a complete solution with live or simulated data and observed stable operation.
- Verified that the user can interpret outputs: KPI cards, charts, prediction panels, alerts log and settings controls.

6.7 Installation

The installation process follows the setup already defined in the project documentation. The main prerequisites are Node.js, pnpm, Python 3.11+, uv and Arduino IDE. After installing the application dependencies, the Prisma client is generated and the database is created. The machine learning models can then be trained and exported if necessary, environment variables are configured (including SMTP and WhatsApp values) and the ESP32 firmware is uploaded. Finally, the application server is started and the dashboard is opened in the browser.

- Install Node.js, pnpm, Python and Arduino IDE.
- Install application dependencies.
- Generate Prisma client and create the database.
- Train and export machine learning models if needed.
- Configure the .env.local file (database, SMTP and WhatsApp variables).
- Upload the firmware to the ESP32 board.
- Start the Next.js server.
- Open the dashboard in the browser.

6.7.1 User training

Basic user training is necessary so that the intended user can start the system, interpret the dashboard, review predictions and respond appropriately to alerts. For this prototype, training would focus more on system use and interpretation of monitoring outputs than on programming or low-level configuration.

6.7.2 System conversion

Because the current monitoring approach at HIT is fragmented and largely manual, the prototype would be introduced as a new monitoring layer rather than as a direct replacement of an existing integrated system. This means conversion would be gradual. The prototype would first operate as

a pilot installation so that its outputs can be compared with the existing approach before wider deployment.

6.7.3 File conversion

There is very little legacy file conversion required for this prototype because the system creates its own SQLite database and stores fresh readings from the sensor node. If HIT later decides to import historical solar records from another source, that can be handled as a future extension. At prototype stage, file conversion mainly involves database initialization, model export files and environment configuration.

6.7.4 System changeover strategy

The most suitable changeover strategy for this project is pilot changeover. This is because the prototype is new and should first be tested on a small monitored setup before being trusted at larger institutional level. A pilot strategy reduces risk, makes it easier to compare performance with existing monitoring practices and gives time for staff to become familiar with the system.

6.8 Conclusion

This chapter explained how the proposed system was implemented and how it can be tested. It showed that the project was developed in modules covering firmware, dashboard, backend, database and machine learning. It also outlined the testing approach, installation procedure, user training needs and suitable changeover strategy. Together, these show that the system is not only conceptually sound but also practical as a working prototype.

CHAPTER 7: CONCLUSION

The study set out to address the weakness of reactive and fragmented solar monitoring at the Harare Institute of Technology by proposing an AI-enhanced IoT monitoring solution. Across the previous chapters, the report established the background of the problem, reviewed related work, analyzed the current system, designed the proposed solution and outlined how the prototype can be implemented and tested. The result is a system that combines sensing, storage, analytics, dashboard visualization and alerting into one monitoring platform suited to the HIT context.

The project demonstrates that combining IoT and machine learning can improve solar monitoring by moving from simple observation to intelligent interpretation. Instead of depending only on manual inspection or static thresholds, the proposed solution makes use of real-time telemetry, prediction models and automated alerts in order to support earlier intervention and better maintenance planning. At prototype level, this provides a useful foundation for future expansion to larger institutional arrays. (Kim and Kim, 2021; Abdel-Nasser and Mahmoud, 2019)

7.1 Scope for Future Extension

Although the current system works as a prototype, there is still room to improve and expand it. One obvious next step is to move from a single 15W prototype panel to larger campus installations at HIT. The same monitoring concept can be applied to multiple panels or full arrays so that the institution can have more complete visibility across its solar infrastructure.

Another possible extension is adaptive retraining of the machine learning models. As more real operational data becomes available, the models can be retrained using local HIT or Zimbabwean data rather than relying only on synthetic prototype data. Future versions can also move further toward hybrid cloud-edge operation so that prediction remains available even when connectivity is unstable. (Li, Sun and Li, 2022)

7.2 Maintenance

The system will require regular maintenance if it is to remain useful over time. This is especially important because the project combines hardware sensors, firmware, web software, a database and machine learning models. A problem in any one of these layers can affect the quality of monitoring. Regular maintenance therefore helps ensure that the system stays accurate, reliable and secure. (Zonta et al., 2020)

7.2.1 Interval System Review

A routine review schedule should be followed after deployment. Daily or weekly checks should confirm that the ESP32 is still sending data and that alerts are working. Monthly checks should review dashboard trends, sensor stability and database growth. Quarterly checks should review model behaviour, false alerts and physical wiring. A yearly review should confirm whether the system still matches operational needs and whether upgrades are necessary.

7.2.2 Maintenance Activities

- Sensor recalibration when readings drift over time
- Checking ESP32 Wi-Fi configuration and server address
- Updating web application dependencies
- Backing up the SQLite database
- Checking that ONNX model files are present and still valid
- Reviewing alert thresholds and SMTP settings
- Retraining models when more representative data becomes available
- Replacing damaged or unstable hardware components when necessary

7.2.3 Disaster Recovery

A simple disaster recovery plan is important even for a prototype. If the system fails, the key items that need recovery are the application code, database file, environment settings and model files. Since the project uses SQLite, a backup of the database file is especially important. In case of failure, the system can be restored by reinstalling dependencies, restoring the database backup, restoring the model files and reconfiguring the environment variables. The ESP32 firmware can also be reflashed if necessary.

7.3 Recommendations

- HIT should consider piloting the system on an actual campus solar installation after prototype validation.
- More real operational data should be collected so that the prediction models can be retrained on local conditions.
- Future versions should include stronger deployment security such as authentication, HTTPS and role-based access control.

- The system should be expanded gradually rather than all at once, starting with one monitored array.
- Staff who will use the system should receive basic training on dashboard interpretation and alert response.
- The institution should treat the system as a decision-support tool and continue to combine it with practical engineering judgement.